Lab 01

→ Vernam One Time Pad:

$P = \{P_1, P_2, \dots, P_n\}$ ← all bits $\in \{0,1\}$

$K = \{K_1, K_2, \dots, K_3\}$

↳ $enc(P) = [P_1 \oplus K_1, \dots, P_n \oplus K_n] = C$

↳ $dec(C) = [C_1 \oplus K_1, \dots, C_n \oplus K_n] = P$

→ Addition Mod p cipher:

1) Pick p larger than space of Msg.
2) Pick randomkey K in $[0; p-1]$

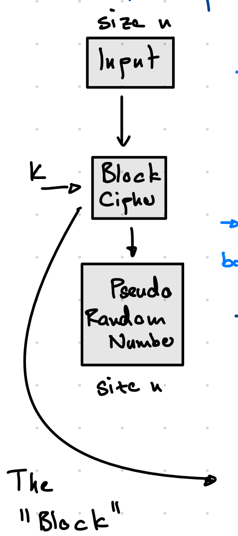
→ $enc(P) = (P + K) \% p = C$

→ $dec(C) = (C - K) \% p = P$

→ Attacks on ciphers:

1. Ciphertext-only
2. Known Plaintext
3. Chosen Plaintext
   ↳ Attacker can feed own P examples into cipher

→ Block Cipher

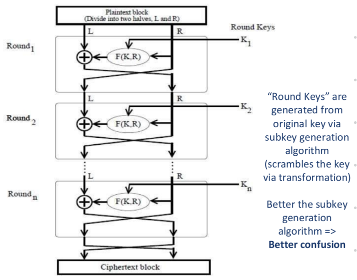


→ Diffusion: arbitrarely small change in input causes new PRN in output

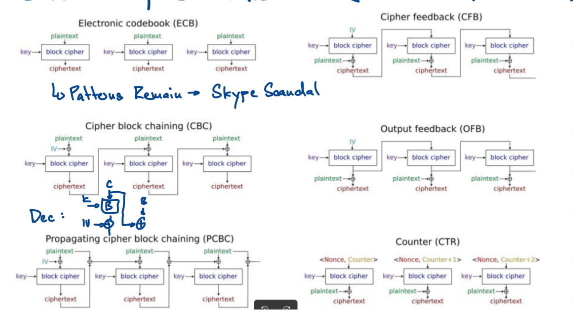
→ Confusion: " same for key broken (only 56 bit key)

→ DES / AES ← keysize 128

→ Feistel Network

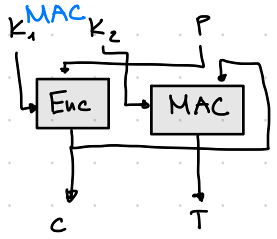


→ Block Cipher Modes: (Blocks of size 128)



Lab 02

Encrypt then MAC



Common Mac:

$HMAC(K, m) = H((K \oplus opad) \| H((K \oplus ipad) \| M))$

$C^* = (C, T)$

→ Birthday Paradox: $n \approx \sqrt{|m|}$ ⇒ $P_{Collision} \approx 50\%$

→ Group ($\mathbb{Z}_{P_1}^*$, * mod p)

↳ all nonzero numbers mod p multiplied and reduced mod p

↳ Generator: Element of the group that can generate all members of the group just by applying the group operation.

→ Characteristics of a Group: @ = Group operator

closure: $\forall a, b \in G: a @ b \in G$

Associativity: $(a@b)@c = a@(b@c)$

Identity: $\exists I$ s.t. $\forall a \in G: a @ I = a$

Inverse: $\forall a \in G\ \exists a^{-1}$ s.t.: $a^{-1} @ a = I$

→ A group is abelian if the order of an operation does not matter (Commutativity)

$a \times b \bmod p = b \times a \bmod p$

→ A group is cyclic if repeatetly aplying the group operation to one elmt, creates entire group

$\exists g \in G$ so that $G = \{g^k \bmod p,\ k \in \mathbb{Z}\}$

→ Square & Multiply: compute $a^e \pmod n$

1) compute e in binary $k_s, k_{s-1}, \dots, k_0$
2) for $i = 0$ to s:
   calculate $b = a^{2^i} \bmod n \equiv (b^{-1})^2 \bmod n$
3) for every b_s if $k_s = 1$:
   $a^e \bmod n \equiv \prod b_s \bmod n$

solve $a^e \bmod n$ $7^{2025} \bmod 91$

1) get bin(e):

→ 0111 1110 1001

2025 % 2	:	1
1012 % 2	:	0
506 % 2	:	0
253 % 2	:	1
126 % 2	:	0
63 % 2	:	1
31 % 2	:	1
15 % 2	:	1
7 % 2	:	1
3 % 2	:	1
1 % 2	:	1
0 % 2	:	0

2) calculate $a^{2^b} \bmod n$ for b in (0, len(bin(e)))

1	$7^1 \equiv 7 \bmod 91$	: 7
0	$7^2 \equiv 7^2 \bmod 91$	: 49
0	$7^4 \equiv 49^2 \bmod 91$	: 35
1	$7^8 \equiv 35^2 \bmod 91$	: 42
0	$7^{16} \equiv 42^2 \bmod 91$	: 35
1	$7^{32} \equiv 35^2 \bmod 91$	: 42
1	$7^{64} \equiv$	: 35
1	$7^{128} \equiv$	: 42
1	$7^{256} \equiv$	: 35
1	$7^{512} \equiv$	: 42
1	$7^{1024} \equiv$	: 35
0	$7^{2048} \equiv$	: 42

3) sum a^{2^b} if $k_{s=b} = 1$

$7^{2025} \bmod 91 \equiv 7^1 \times 42^4 \times 35^3 \bmod 91$

$= 21$

Lab 3

→ Diffie-Hellman Key Exchange:

P, g, A

P, g, B

Entity A

Prime P

Generator of $(\mathbb{Z}_{P_1}^*, *)$ g

$a \in \mathbb{Z}_P^*$ — private

Public key $A = g^a \bmod p$

$K_a = B^a$

↳ $K_a = B^a = (g^b)^a = (g^a)^b = A^b = K_b$

Entity B

Prime P

Generator of $(\mathbb{Z}_{P_1}^*, *)$ g

$b \in \mathbb{Z}_P^*$

Public key $B = g^b \bmod p$

$K_b = A^b$

→ DHKE security: Attacker wants to use $A = g^a$ and $B = g^b$ to compute g^{ab}

↳ NP-hard problem

→ DHKE on its own is vulnerable against Man-In-The-Middle Attacks

→ Group $G = (\mathbb{Z}_N^*, *)$ really $G = (\mathbb{Z}_N^*, * \bmod N)$

$G = \{x \in \{1, 2, \dots, n-1\} \mid gcd(x, n) = 1\}$

↳ Set with positive Integers to N-1 that are coprime with N

↳ coprime: $gcd(x, N) = 1$

↳ Abelian Group

→ Extended Euclidean Algo to find gcd:

```
gcd(a, b):
  while b != c:
    a, b = b, a % b
  return a
```

→ Order of Integer a: smallest m s.t. $a^m \equiv 1 \bmod n$

→ Order of group $\mathbb{Z}_n^*$ $\phi(n)$: total count of numbers up to n which are coprime to n

↳ Eulers Totient: $\phi(n) \geq ord(x)\ \forall x \in \mathbb{Z}_n^*$

↳ Lagrange Theorem: for each $g \in G$: $ord(g)$ divides $\phi(G)$

multiplikativ

$$\boxed{\begin{array}{l} b^{\phi(n)} \equiv 1 \bmod (n) \quad \forall b \in \mathbb{Z}_n^* \\ \phi(n) = ord(b) \times k \\ b^{\phi(n)} = b^{ord(b) \times k} \end{array}}$$

→ RSA

Eulers Totient:

if $gcd(m, n) = 1$:

$\phi(mn) = \phi(m)\phi(n)$

Encription:

$C = M^e \bmod n$ $(M<n)$

$P = C^d \bmod n$

Entity Receiver

primes p, q

$n = p \times q$

$\phi = (p-1)(q-1)$

$e = 1 < e < \phi(n) \mid gcd(\phi(n), e) = 1$

$d = e^{-1} \bmod \phi(n)$

↳ $de \equiv 1 \bmod (\phi(n))$

$K_{Pub} = \{e, n\}$

$K_{Priv} = \{d, n\}$

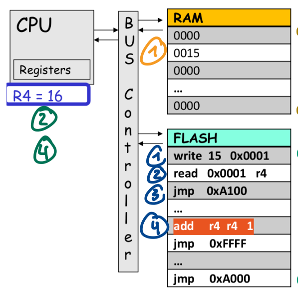
→ RSA Sign = $H(M)^{K_{Priv}} \bmod N$

↳ Offers Non-Repudiation: cant deny creating the Sign since no one else can

Lab 4

RAM address range: 0x0000 to 0x9FFF

FLASH address range: 0x0000 to 0x9FFF



→ Computer Instructions

Boot => next(PC) = A000

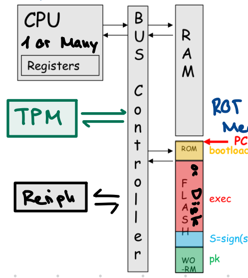
(PC = A000) =>
Daddr = **0x0001** ; Ren = **0** ;
Wen = **1** ; next(PC) = **0xA001**

(PC = A001) =>
Daddr = **0x0001** ; Ren = **1** ;
Wen = **0** ; next(PC) = **0xA002**

(PC = A002) =>
Daddr = **??** ; Ren = **0** ;
Wen = **0** ; next(PC) = **0xA100**

(PC = A100) =>
Daddr = **??** ; Ren = **0** ;
Wen = **0** ; next(PC) = **0xA101** 53

Secure Boot Archi



- **Program Memory** now has:
  - FLASH: ← In general purpose devices, → Disk
    - Stores the **"operative runtime system":** e.g., OS (if any), apps, etc.
    - A part of Flash (fixed location) reserved to store a code authorization token S
    - Bootloader will search for S in that fixed location.
    - Only the device owner can produce valid S, because only the device owner knows sk corresponding to pk stored in WO-RM/PROM.

→ Boot of a PC:

1 - User switches on the system; power-supply does a self-check and send "Power-Good Signal" to the CPU

BIOS: Basic Input Output System

2 - CPU starts executing code at address ***0xFFFF0***, which is in **BIOS ROM**

3 - BIOS code performs additional hardware tests and searches for bootable devices

4 - On finding a bootable device, the BIOS loads its **boot sector** to main memory and executes it.

MBR: Main Boot Record = Harddrive

5 - The MBR code checks the partition table (within the bootable device) for an active partition.

- If (only) one is found, the MBR code loads that partition's boot sector and executes it.

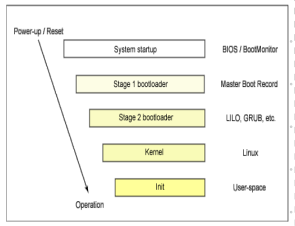
- Otherwise, MBR may load a secondary bootloader that often allows the user to choose a partition to load.

6 – Secondary bootloader loads the OS Kernel from the partition of choice and executes it.

Done!

One possible secure boot chain:

1 – BIOS verification code is unmodifiable (ROM) and trusted
2 – BIOS ***verifies*** Stage 1 bootloader (MBR) on chosen boot device
3 – BIOS loads and runs verified MBR
4 – MBR ***verifies*** Stage 2 bootloader (GRUB)
5 – MBR loads and runs GRUB
6 – GRUB ***verifies*** selected OS partition
7 – GRUB loads and runs selected OS partition
8 – OS loads and runs applications



→ Rollback attack: Starting a device with old exec in Flash and Sign(sk, exec)

↳ Prevention: Sign(sk, counter || exec) and bootloader keeps track of counter

↳ Store in counter region to which only the bootloader can write

→ Remote Attestation: Check if a device is running on software that it should run on

Verifier

1) choose random nonce chall

3) compare

$MAC(K, PMEM \| chall)$

$==$

$MAC(K, ExpectedSW \| chal)$

Prover (device)

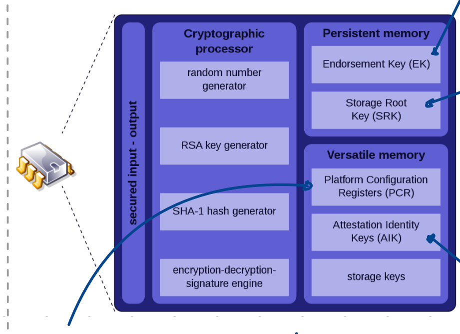
chall

2) Root of Trust in Device ROT computes $MAC(K, PMEM \| chall)$

↳ Device is live & Devices ROT knows K

(Proving Authenticity of measured state)
(never leaves TPM)

ROT for Reporting



ROT for Storage (used to encript secrets on disk)

Generated to sign Measured PCR Values. Certified

Store cryptographic measurements of a systems state

↳ chaine that can only be extended

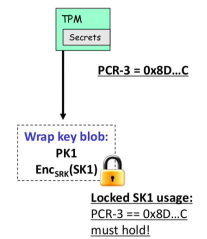
$PCR_{0,i} = H(PCR_{0,i-1} \| input)$

State-Based Decryption:

Encrypt some data that can only be decrypted at a latter time if the system is in a known "good state".

Known value of PCR-3 after **"good boot"**

- Generate new key pair
- At key generation time, impose Restriction:
  "Hey TPM, you should only use this key when PCR-3 = 0x8D...C"
- **Encrypt the data using the produced public key (PK1)**



Lab 5

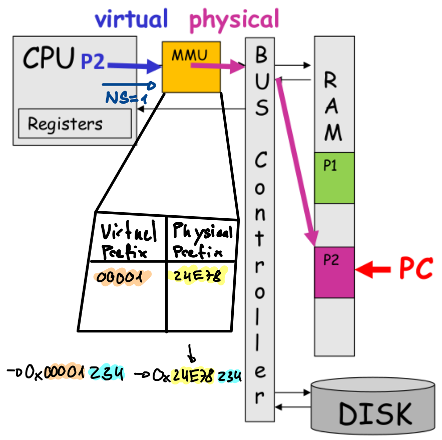
→ Runtime System Isolation
- ↳ Ensured by OS
- ↳ Each process is assigned a given mem space
- In Embedded ↳ Memory Protection Unit (MPU) - blocks access
- In general purpose devices ↳ Memory Management Unit (MMU) - engine to support virtual Memory
  - ↳ Pagetable is created for process whenever a new process is spunup
  - ↳ In multicore Processors → each core has its own MMU

→ System calls → Process gives subtask to which it has no privilege to OS

→ Trusted Computing Base (TCB): OS with full access on entire system / memory space
- ↳ Huge → many possible bugs

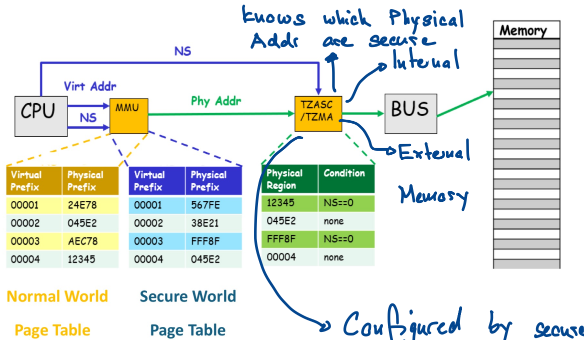
→ Trusted Exec Environments (TEE): Architectures to reduce size of TCB
- ↳ Programmable
- ↳ ARM Trustzone

→ MMU mapping:



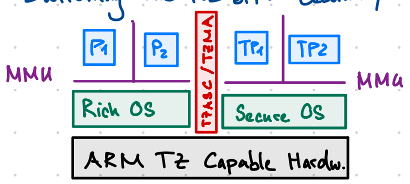
↳ MMU has two tables:
- → One for Normal World → NS = 1
- → One for Trusted World → NS = 0

knows which Physical Addr are secure

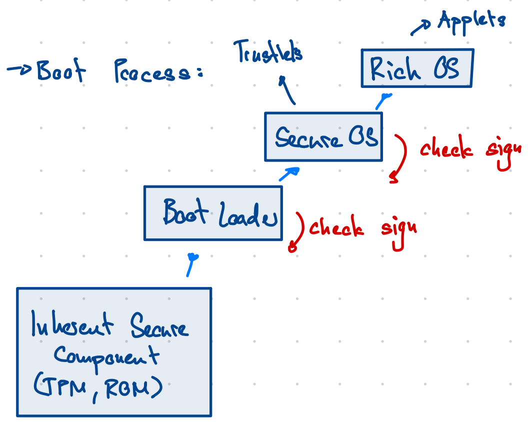


→ Configured by secure OS during boot

→ Switching the NS bit: Security Monitor

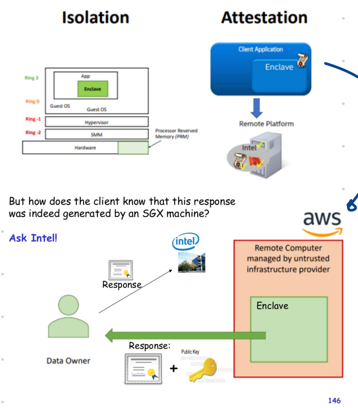


→ Boot Process:



RoT

→ Intel SGX: works with enclaves



Remote Attestation is often used in cloud computing
↳ giving personal data to a Enclave on a unknown machine

→ Three Main causes for Vulnerabilities:
1) Rush to Market
2) Ignoring weakest link
4) Complex Software